

Problem Solving Strategies and Computational Approaches SCS1304

Handout 6 : Dynamic Programming

Prasad Wimalaratne PhD

Overview

- Introduction to Dynamic Programming (DP)
- Divide Conquer vs Dynamic Programming
- How Does DP Work?
- Examples of DP
- When to use DP?
- Approaches to DP
 - Tabulation
 - Memoization
- Ref: 4 Principle of Optimality –
- [Dynamic Programming introduction](https://www.youtube.com/watch?v=5dRGRueKU3M)
<https://www.youtube.com/watch?v=5dRGRueKU3M>

DP Real-Life Example: The Jigsaw Puzzle Analogy

- Dynamic programming can be understood with **the analogy of solving a jigsaw puzzle** efficiently. Imagine you are solving a large jigsaw puzzle. Instead of trying random pieces to see what fits (brute force), you use a systematic approach:



- Break It Down:** You group the puzzle pieces by color or pattern (e.g., edges, sky, trees, etc.), making it easier to focus on smaller parts of the puzzle.
- Reuse Your Work:** Once you solve a smaller section (e.g., the sky), you do not revisit it or redo it. Instead, **you place it aside, knowing it is solved**. This is similar to memoization in dynamic programming.
- Build Step by Step:** After solving smaller sections, you **combine them to form larger sections** (e.g., attaching the sky to the trees). This is similar to tabulation, where you **solve subproblems iteratively and build up to the final solution**.
- Avoid Redundancy:** If you have already figured out where a piece fits, you do not test it again in other places. This prevents repetitive work and saves time.

Dynamic Programming

Paradigm

**Divide
and
Conquer**

Overlapping
Subproblems

AND

Optimal
Substructure

+

Methodology

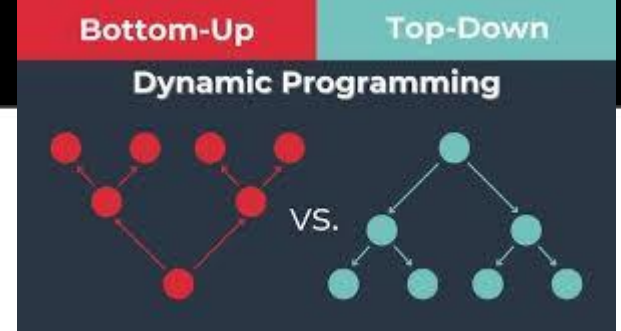
Memoization

↓ Top-down approach

OR

Tabulation

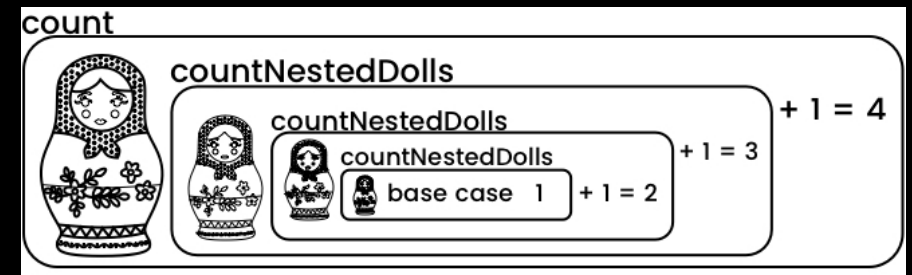
↑ Bottom-up approach



Dynamic programming

- Dynamic programming is an algorithmic technique used to **solve complex problems by breaking them down into simpler, overlapping subproblems**.
- The core idea is to **avoid redundant computations by storing the solutions** to these subproblems and reusing them when needed.
- This approach is particularly effective for **optimization problems**, where the goal is to find the **best possible solution** (e.g., **minimum cost, maximum value**).
- A technique where an algorithmic problem is first **broken down into sub-problems**, the **results are saved**, and then the sub-problems are optimized to find the overall solution which **usually has to do with finding the maximum and minimum** range of the algorithmic query.
- The presence of **overlapping subproblems** is **a key characteristic that signals the applicability and effectiveness of dynamic programming**.
- Richard Bellman was the one who came up with the idea for dynamic programming in the 1950s.

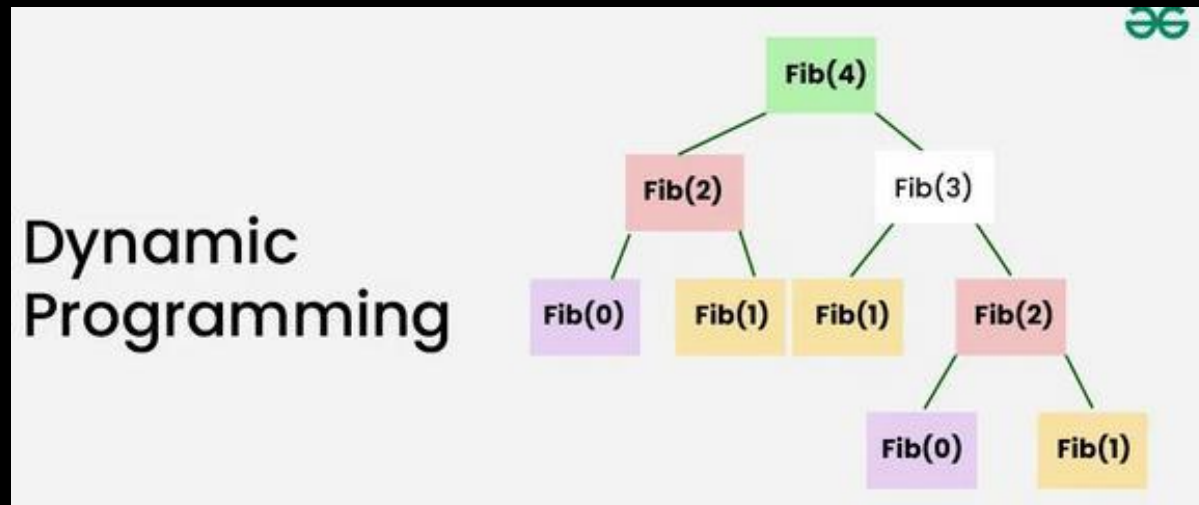
Dynamic programming



- Recursion from previous lesson -> the solution to a problem **depends on solutions to its smaller subproblems**.
- Dynamic programming is an optimization technique that improves the efficiency of recursive OR iterative solutions **by storing and reusing** the results of **overlapping subproblems**. (i.e. repeatedly solve the same subproblems (overlapping subproblems))
 1. It is the preferred technique for **solving recursive functions that make repeated calls to the same inputs**.
 2. However, ****NOT all problems that use recursion can be solved by dynamic programming**.
- ****Unless solutions to the subproblems overlap, a recursion solution can only be arrived at using a divide-and-conquer method**.
 - For example, problems like merge, sort, and quick sort are **NOT** considered dynamic programming problems.

Dynamic Programming

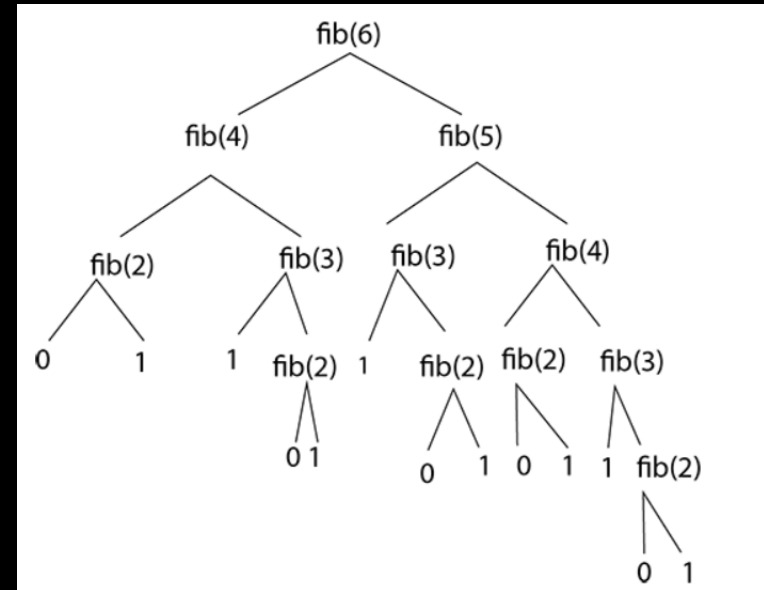
- Dynamic Programming is **a method used in mathematics and computer science** to solve complex problems by breaking them down into simpler subproblems.
- By **solving each subproblem only once and storing the results**, it **avoids redundant computations**, leading to more efficient solutions for a wide range of problems.
- Note the same colors in diagram below



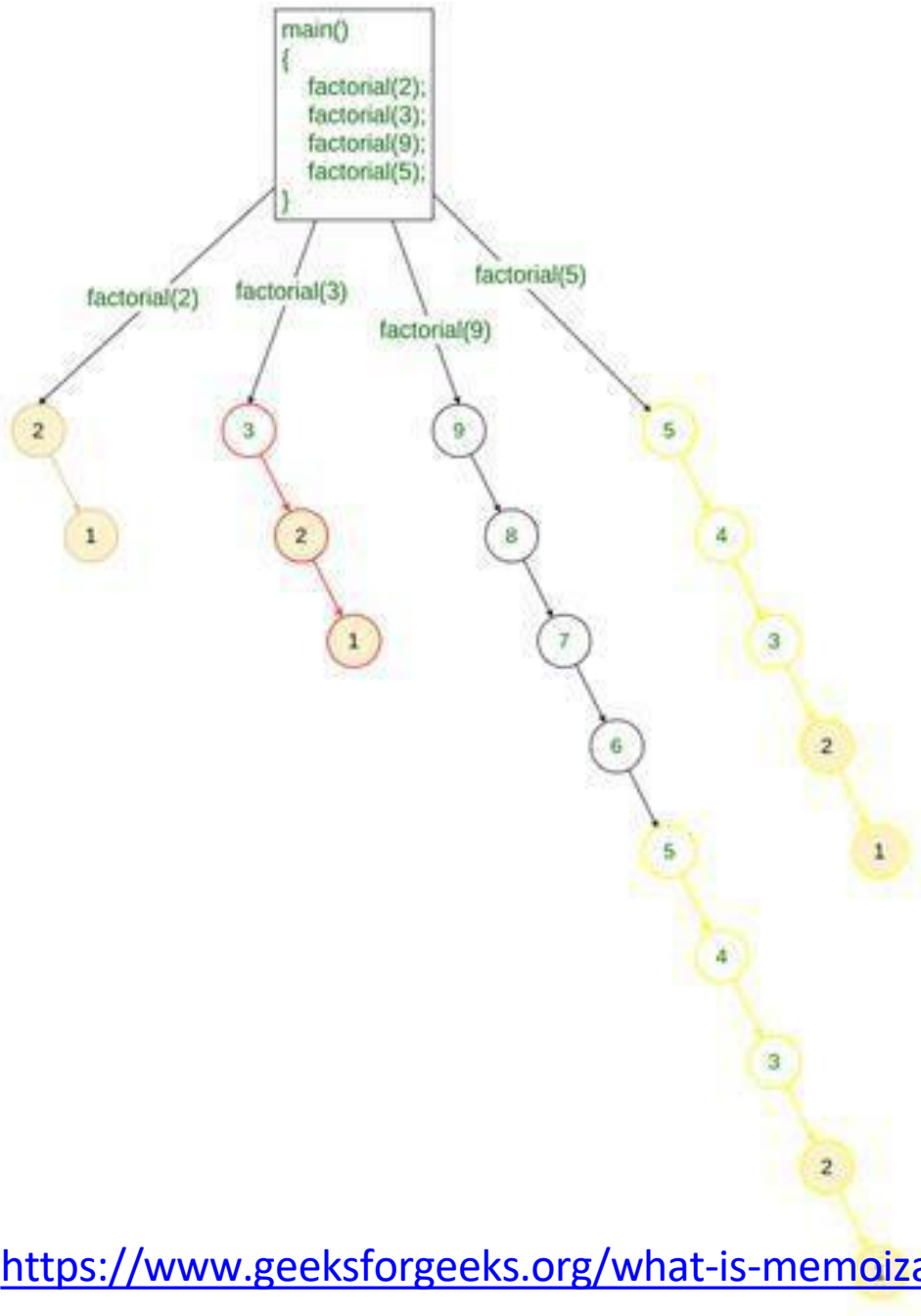
Fibonacci

If you are asked to compute the 50th Fibonacci number using the recursive definition $F(n) = F(n-1) + F(n-2)$, How long would that take?

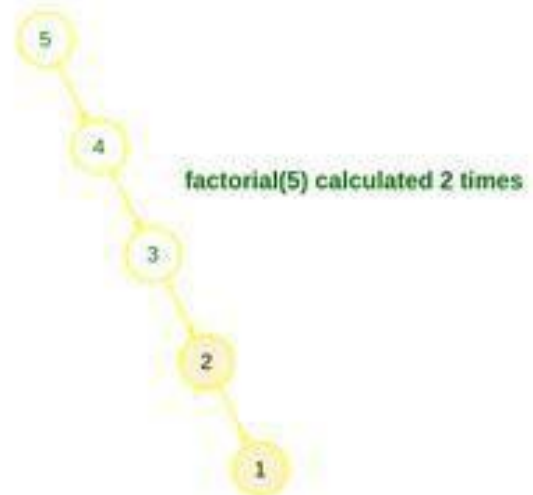
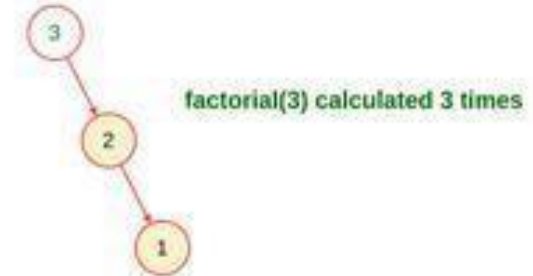
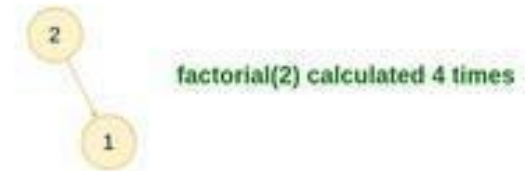
recursion tree for $F(6) \Rightarrow$



$F(3)$ is computed **three times**. $F(2)$ is computed **five times**. As n grows, this increases exponentially



factorial(2)
factorial(3)
factorial(9)
factorial(5)



Key Differences Between Divide and Conquer and Dynamic Programming

- Approach:
 - Divide and Conquer: Breaks a problem into smaller subproblems and solves them independently.
 - Dynamic Programming: Breaks a problem into overlapping subproblems and solves them recursively OR iteratively, storing their solutions.
- Subproblem Dependency:
 - Divide and Conquer: Subproblems are independent of each other.
 - Dynamic Programming: Subproblems overlap or share common subproblems.
- Solution Utilization:
 - Divide and Conquer: Does not typically utilize previously solved subproblem solutions.
 - Dynamic Programming: Utilizes previously solved subproblem solutions by storing them for reuse.
- Solution Construction:
 - Divide and Conquer: The solution is obtained by combining the solutions of independent subproblems.
 - Dynamic Programming: The solution is built from the stored solutions of smaller overlapping subproblems iteratively OR recursively.

Key Difference Between Divide and Conquer and Dynamic Programming ctd..

- Time Complexity:
 - Divide and Conquer: Time complexity **depends on the algorithm and the problem**. It **does not store subproblem solutions, so repeated computations may occur if subproblems overlap**.
 - Dynamic Programming: Typically takes a shorter time to solve the problem.
- Subproblem Nature:
 - Divide and Conquer: Works well when subproblems are independent.
 - Dynamic Programming: Works well when subproblems overlap or share common subproblems.
- Example Algorithms:
 - Divide and Conquer: **Merge Sort, Quick Sort**.
 - Dynamic Programming: **Fibonacci series, Knapsack problem**.
- Memory Usage:
 - Divide and Conquer: Can be more memory-efficient in certain cases.
 - Dynamic Programming: May require more memory due to the storage of subproblem solutions.
- Approach Complexity:
 - Divide and Conquer: Generally simpler to understand and implement.
 - Dynamic Programming: Can be more complex to understand and implement.

DP

[1] CORE CONCEPT: Break & Reuse



BIG PROBLEM



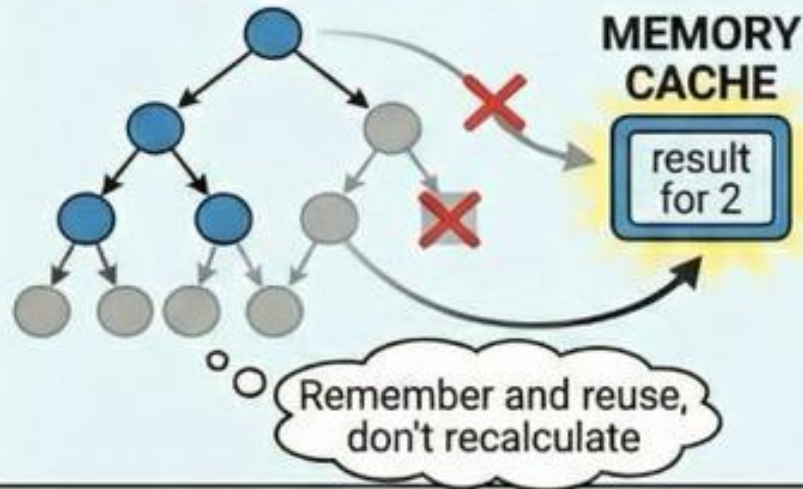
Combine answers
to small for
BIG ONE



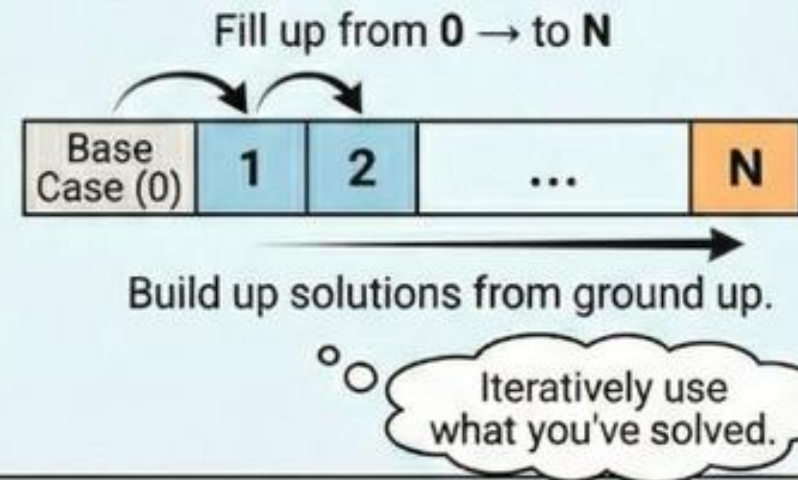
Optimal Substructure:
Build big from small answers.

[2] THE KEY: Avoid Repeating Work

MEMOIZATION (Top-Down)



TABULATION (Bottom-Up)



How Does Dynamic Programming (DP) Work?

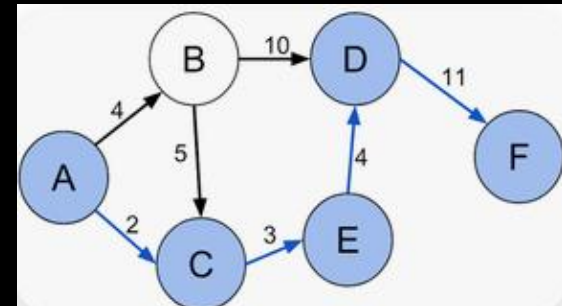
Dynamic programming is an optimization technique used when solving problems that consists of the **following characteristics**:

1. **Optimal Substructure**: The optimal solution can be built from optimal solutions to smaller subproblems.
 - To **solve a large problem optimally**, you can solve **smaller subproblems optimally** and combine their results.
2. **Overlapping Subproblems**: The same subproblems occur multiple times, so storing their solutions saves computation.

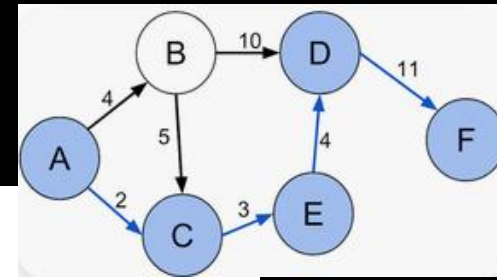
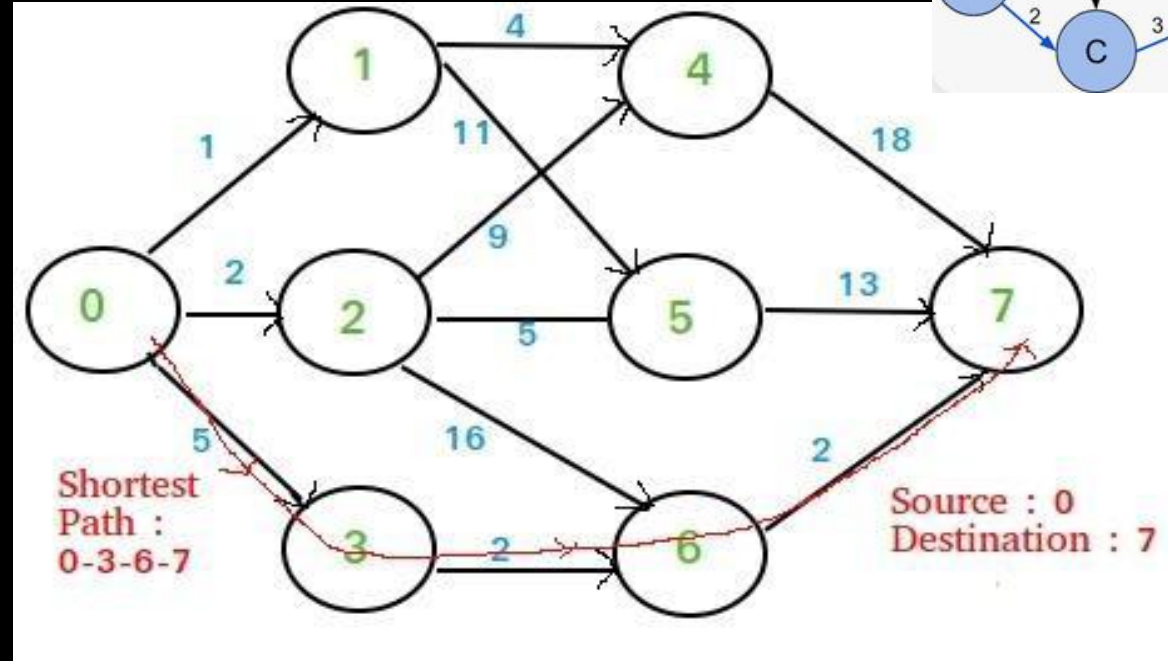
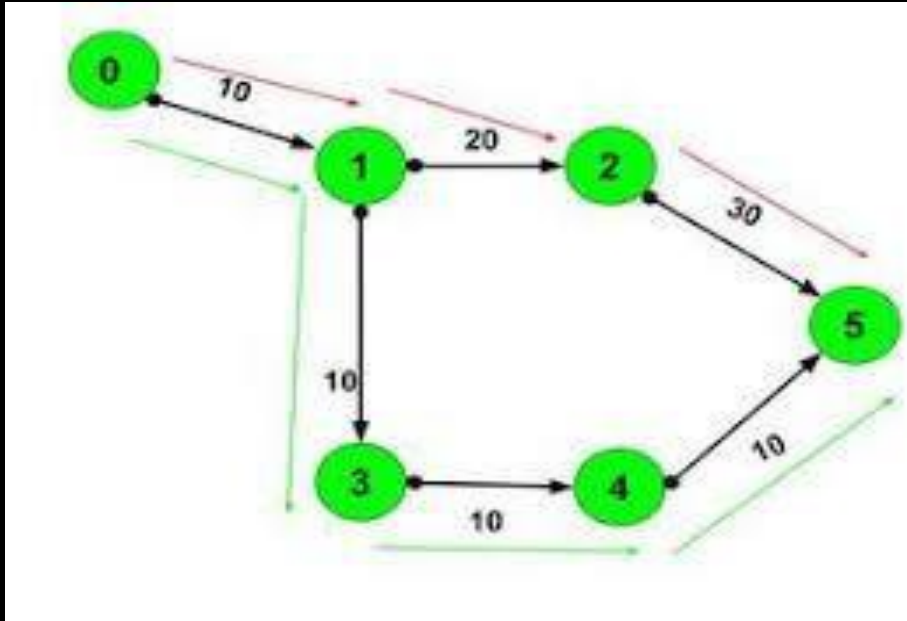
When to Use Dynamic Programming (DP)?

- **1. Optimal Substructure:**

- Optimal substructure means that we combine the optimal results of subproblems to achieve the optimal result of the bigger problem.
- Example: Consider the problem of finding the minimum cost path in a weighted graph from a source node to a destination node.
- We can break this problem down into smaller subproblems:
 1. Find the **minimum cost path** from the **source** node to each **intermediate node**.
 2. Find **the minimum cost path** from each **intermediate node** to the **destination node**.
- The solution to the larger problem (finding the minimum cost path from the source node to the destination node) can be constructed from the solutions to these smaller subproblems.



What is Shortest Path?



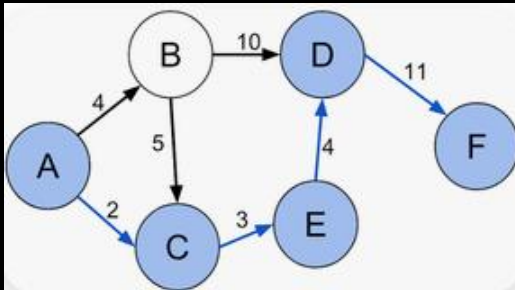
- For example, the Shortest Path problem has the **following optimal substructure property** –
 - If a **node x lies in the shortest path** from a **source node u to destination node v**, then the shortest path from u to v is the combination of the shortest path from u to x, and the shortest path from x to v.
- The standard All Pair Shortest Path algorithms like [Floyd-Warshall](#) and [Bellman-Ford](#) are typical examples of Dynamic Programming.

Optimal Substructure

Optimal substructure means that an optimal solution to a problem can be constructed by **combining optimal solutions of its smaller subproblems**.

For example, in shortest path planning, the shortest path between two locations contains shortest paths between intermediate locations.

- Suppose a robot needs to travel from A to F with the minimum distance.
 - $A \rightarrow C \rightarrow E \rightarrow D \rightarrow F$
 - If **the optimal path from A to D is $A \rightarrow C \rightarrow E \rightarrow D \rightarrow F$**
 - then the subpath: **$C \rightarrow E \rightarrow D \rightarrow F$ must also be the shortest path from C to F.**



Reason?

- If there were a shorter path from C to F, we could replace the current subpath.
- This would create a shorter path from C to F, **contradicting the fact that $A \rightarrow C \rightarrow E \rightarrow D \rightarrow F$**
- **was optimal.**

When to Use Dynamic Programming (DP)? Ctd...

- **2. Overlapping Subproblems:**

- The same subproblems are solved repeatedly in different parts of the problem.
- Example:
 - Consider the problem of computing the Fibonacci series.
 - To compute the **Fibonacci number at index n** , we need to **compute the Fibonacci numbers at indices $n-1$ and $n-2$** .
 - This means that the subproblem of computing the Fibonacci number at index $n-1$ is used twice in the solution to the larger problem of computing the Fibonacci number at index n .

example Overlapping Subproblems: Factorial

$2! = 2 \times 1$	$= 2$
$3! = 3 \times 2 \times 1$	$= 6$
$4! = 4 \times 3 \times 2 \times 1$	$= 24$
$5! = 5 \times 4 \times 3 \times 2 \times 1$	$= 120$
$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1$	$= 720$
$7! = 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$	$= 5040$
$8! = 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$	$= 40320$
$9! = 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$	$= 362880$
$10! = 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$	$= 3628800$

The factorial of zero = 1.

$$0! = 1$$

How Dynamic Programming Works?...

1. Identify Subproblems

- Divide the main problem into smaller, **overlapping subproblems**.

2. Solve and Store Solutions

- Solve each subproblem and **store its result in a table, array, OR memoization cache**.

3. Build the Final Solution

- Use the **stored solutions of smaller subproblems** to construct the solution to **larger subproblems, eventually solving the original problem**.

4. Avoid Redundant Computation

- Since each subproblem is **solved only once and its result is reused**, repeated calculations are avoided, improving efficiency.

Examples of Dynamic Programming (DP)

e.g: Finding the Fibonacci Sequence

Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

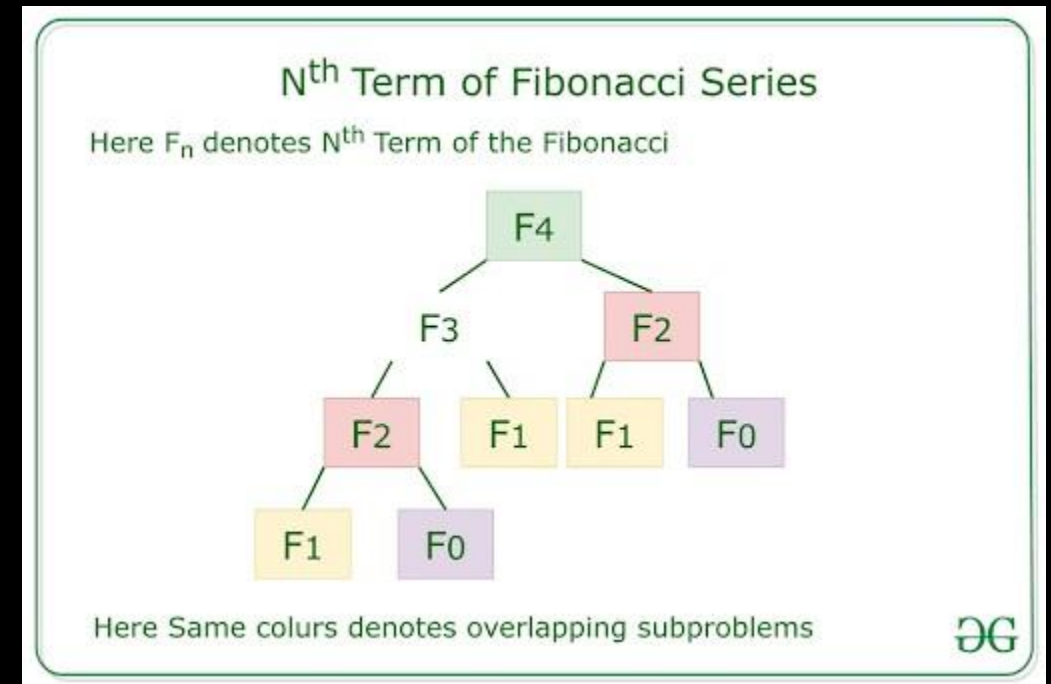
- **Brute Force Approach:** To find the n^{th} Fibonacci number, a recursive algorithm calculates the $(n-1)^{\text{th}}$ and $(n-2)^{\text{th}}$ Fibonacci numbers.
 - This process continues **recursively until the base case** ($F(0) = 0$ OR $F(1) = 1$) is reached.
 - This approach is **inefficient** because it repeatedly recalculates the same Fibonacci numbers.
 - As the **value of n increases**, the number of recursive calls **grows exponentially**, making the algorithm slow for large values of n .

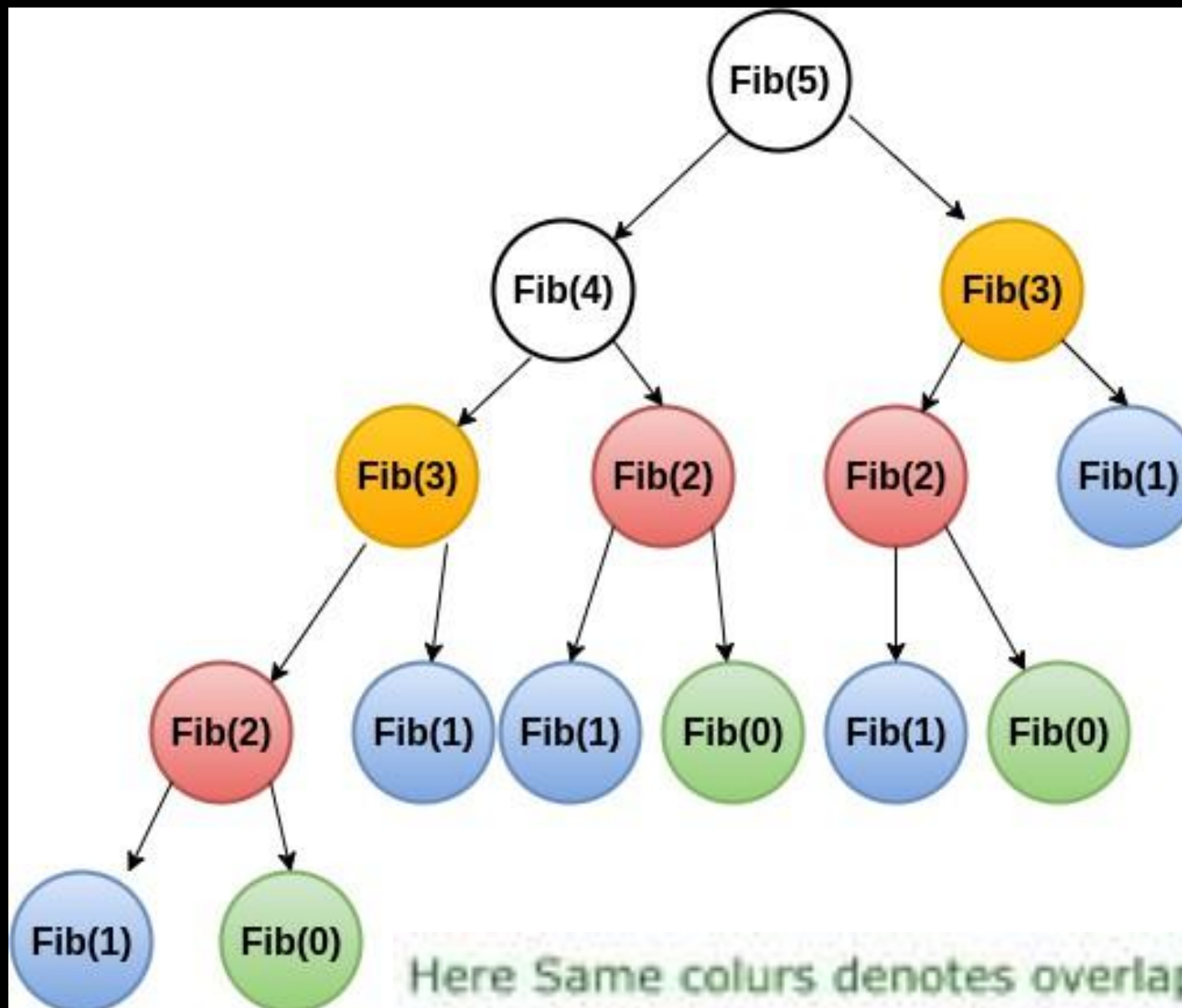
Number of calls to Fib using Brakeforce

i	Fib(i)	numCalls
i = 0	1	1
i = 1	1	1
i = 2	2	3
i = 3	3	5
i = 4	5	9
i = 5	8	15
i = 6	13	25
i = 7	21	41
i = 8	34	67
i = 9	55	109
i = 10	89	177
i = 11	144	287
i = 12	233	465
i = 13	377	753
i = 14	610	1219
i = 15	987	1973
i = 16	1597	3193
i = 17	2584	5167
i = 18	4181	8361
i = 19	6765	13529
... snip ...		
i = 50	20365011074	40730022147

Fibonacci Series using Dynamic Programming

- **Subproblems:** $F(0)$, $F(1)$, $F(2)$, $F(3)$, ...
- **Store Solutions:** Create a table to store the values of $F(n)$ as they are calculated.
- **Build Up Solutions:** For $F(n)$, look up $F(n-1)$ and $F(n-2)$ in the table and add them.
- **Avoid Redundancy:** The table ensures that each subproblem (e.g., $F(2)$) is solved only once.





DP via Memoization and Tabulation

- **Memoization** and **Tabulation** are **two different ways** to solve complex problems faster by **storing answers to smaller parts**.
- Both save time by only doing a calculation once.

Memoization (Top-Down)

- **Start from the original problem**, recurse into subproblems
- **Before computing a subproblem, check a cache (array/hashmap)**
- If cached, return immediately; else compute, store, return
- **'Recursion with a memory'**

Tabulation (Bottom-Up)

- Build a **table starting from the smallest subproblems**
- Fill it iteratively **until reaching the answer to the original problem**
- No recursion, uses loops
- **'Build up the answer piece by piece'**

DP



Memoization

```
function Fibonacci(n):  
    if n in memo:  
        return memo[n]  
    if n <= 1:  
        return n  
    memo[n] = Fibonacci(n-1) + Fibonacci(n-2)  
    return memo[n]
```

Memoization (Top-Down DP)

1. Uses recursion.
2. Solves the problem starting from fib(n).
3. Stores computed values in a memo array to avoid recomputation.

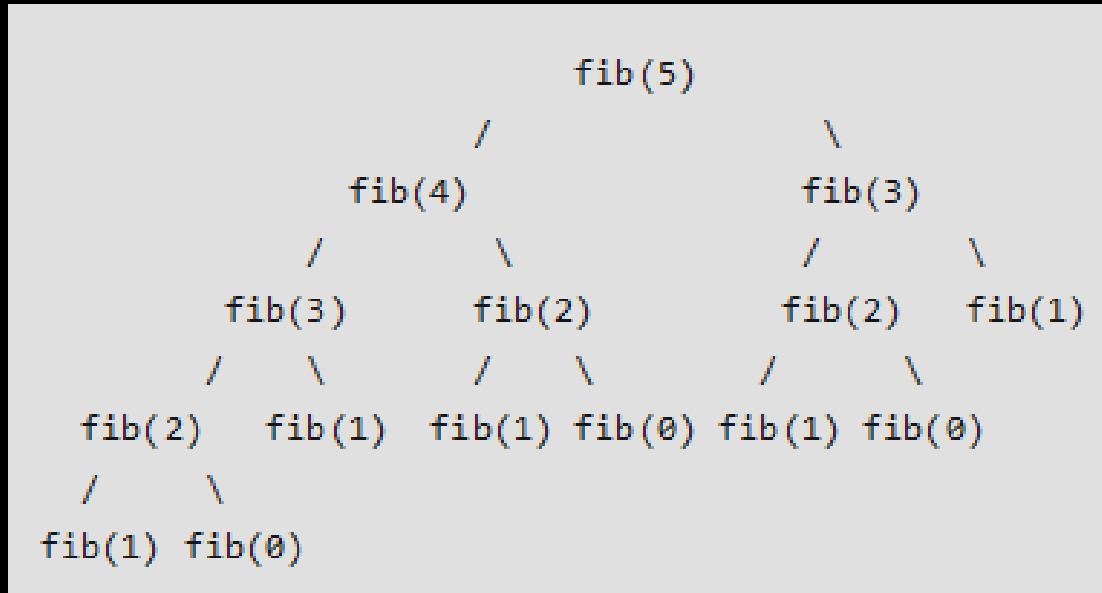
Tabulation

```
function Fibonacci(n):  
    table = array of size n+1  
    table[0] = 0  
    table[1] = 1  
    for i from 2 to n:  
        table[i] = table[i-1] + table[i-2]  
    return table[n]
```

Tabulation (Bottom-Up DP)

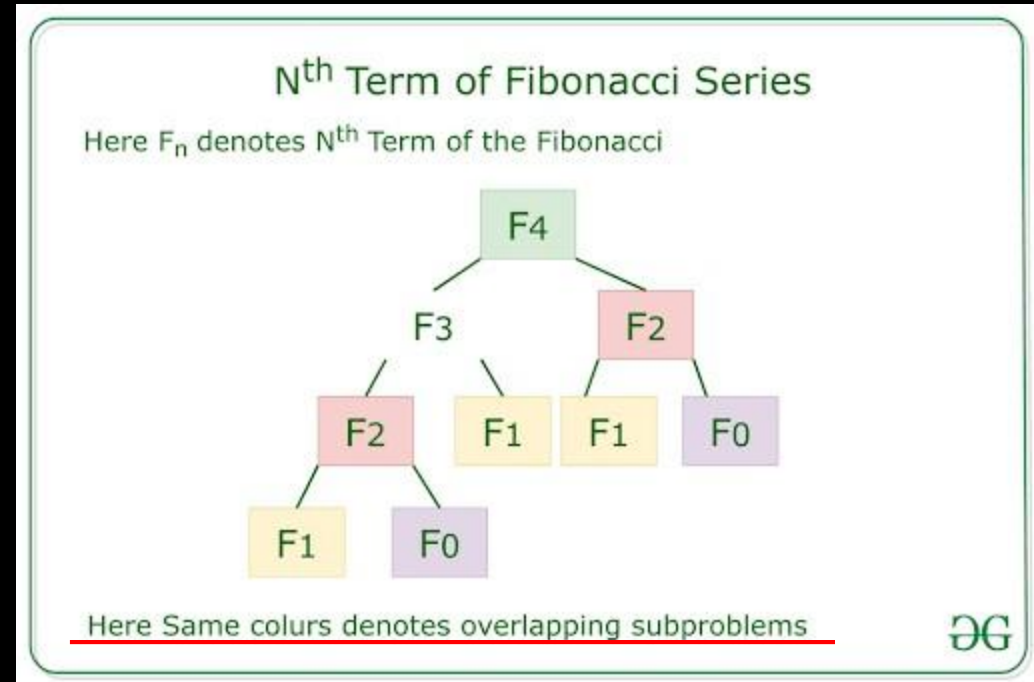
1. Uses iteration (loops).
2. No recursion.
3. Starts from the smallest subproblems and builds up to the answer.

Fibonacci Numbers With and without Dynamic Programming



The same method call is being done multiple times for the same value.

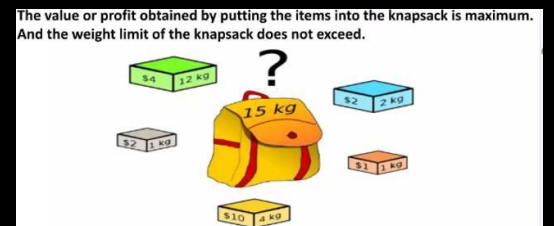
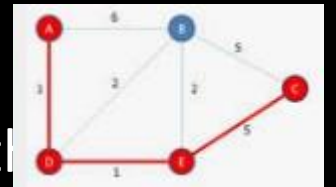
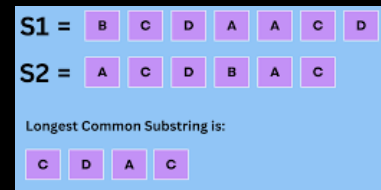
This **can be optimized** with the help of Dynamic Programming.



We can avoid the repeated work done in the Recursion approach by storing the Fibonacci numbers calculated so far.

Example problems for DP

- Example 1: By using DP, we can efficiently calculate the Fibonacci sequence without having to recompute subproblems.
- Example 2: Longest common subsequence (finding the longest subsequence that is common to two strings)
 - Ref : <https://www.youtube.com/watch?v=jHGgXV27qtk>
- Example 3: Shortest path in a graph (finding the shortest path between two nodes in a graph)
- Example 4: Knapsack problem (finding the maximum value of items can be placed in a knapsack with a given capacity)

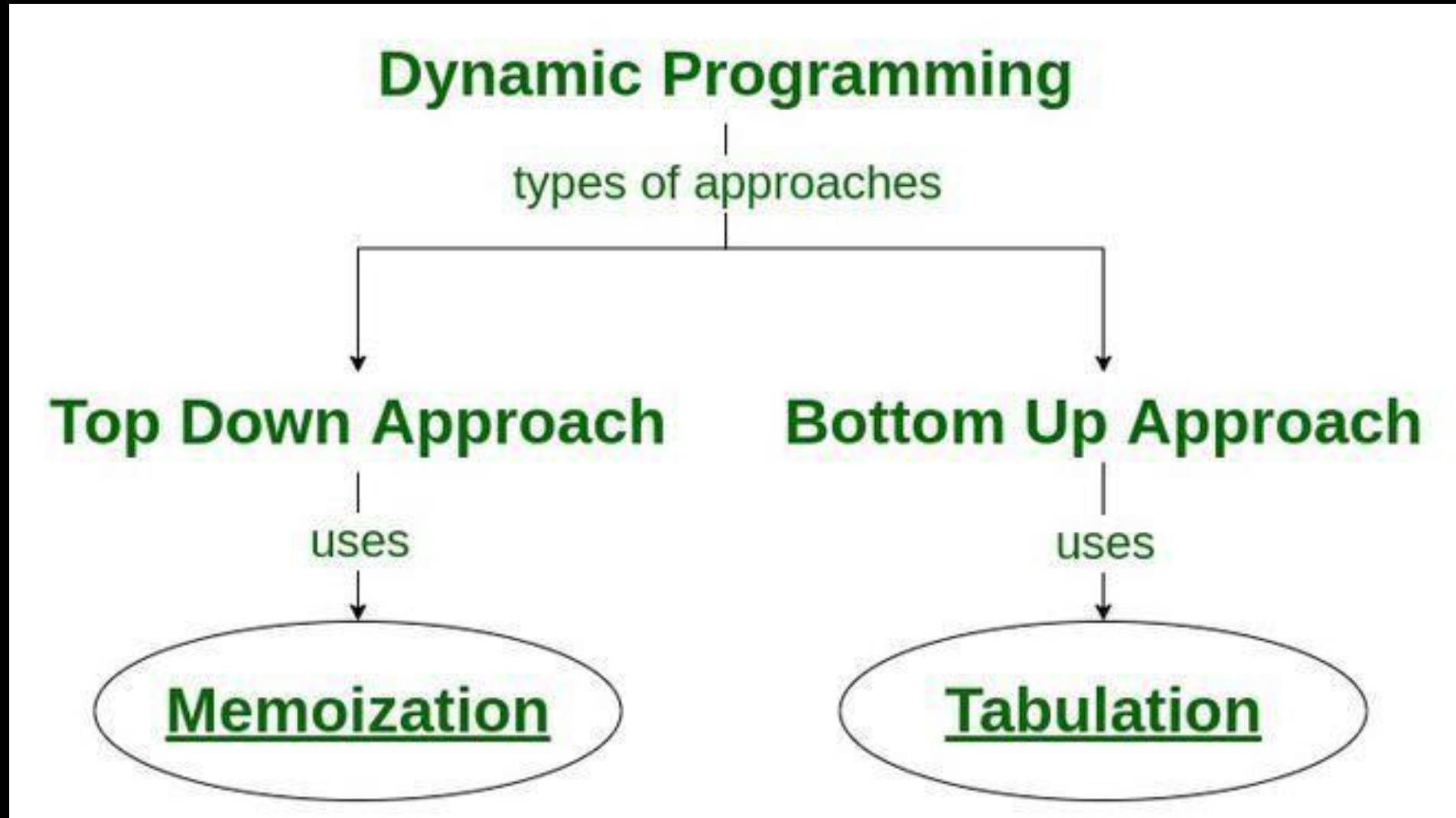


How to Implement DP

There are two main ways to implement DP:

1. Memoization (Top-Down)
 - Uses recursion.
 - Uses memory (cache/memo table).
2. Tabulation (Bottom-Up)
 - Uses memory (a DP table).
 - Usually does not use recursion.

Approaches/Methods of Dynamic Programming (DP)



Approaches of Dynamic Programming (DP)

Dynamic programming can be achieved using two approaches:

- 1. **Top-Down Approach (Memoization)**:
 - In the top-down approach, also known as memoization, we start with the final solution and recursively break it down into smaller subproblems. To avoid redundant calculations, we store the results of solved subproblems in a memoization table.
 - Let us breakdown Top down approach:
 - i. Starts with the final solution and **recursively breaks** it down into smaller subproblems.
 - ii. Stores the solutions to subproblems in **a table to avoid redundant calculations.**
 - iii. Suitable when the number of **subproblems is large and many of them are reused.**

Memoization

- The term “Memoization” comes from the Latin word “memorandum” (to remember), which is commonly shortened to “memo” in American English, and which means “**to transform the results of a function into something to remember.**”
 - storing the result so you can use it next time instead of calculating the same thing again and again
- In computing, memoization is used to **speed up** computer programs by eliminating the repetitive computation of results, and by **avoiding repeated calls to functions that process the same input.**



Why is Memoization used?

- Memoization is a specific form of caching that is used in dynamic programming.
- The purpose of caching is to improve the performance of our programs and keep data accessible that can be used later.
 - It stores the previously calculated result of the subproblem and uses the **stored result for the same subproblem**.
 - This **removes the extra effort to calculate again and again for the same problem**.
- We already know that if the same problem occurs again and again, then that problem is recursive in nature.

Memoization?

- Memoization is a top-down approach to dynamic programming.
- It involves solving a problem by **recursively breaking it down into smaller subproblems** and **storing the results** of these subproblems to avoid recomputing them.
- When a subproblem is solved, its **result is stored in a data structure, typically a hash table or an array**, so that it can be reused if the same subproblem is encountered again.

How Memoization Works

1. **Recursive Function**: The problem is solved using a recursive function that calls itself for smaller subproblems.
2. **Storage of Results**: Before making a recursive call, the algorithm **checks if the result of the subproblem is already stored in the memoization table/cache**. If it is, the stored result is returned **instead of recalculating** it.
3. **Base Case**: The recursive function includes a base case to terminate the recursion.
4. **Store and Return**: If the subproblem result is **not stored, the recursive function computes** it, stores it in the memoization table, and then returns the result.

e.g Memoization

```
function Fibonacci(n):  
    if n in memo:  
        return memo[n]  
    if n <= 1:  
        return n  
    memo[n] = Fibonacci(n-1) + Fibonacci(n-2)  
    return memo[n]
```

Approaches of Dynamic Programming (DP) ctd..

- **2. Bottom-Up Approach (Tabulation):**

- In the bottom-up approach, also known as tabulation, we start with the smallest subproblems and gradually build up to the final solution.
- We store the results of solved subproblems in a table to avoid redundant calculations.
- Let us breakdown Bottom-up approach:
 - i. Starts with the smallest subproblems and gradually builds up to the final solution.
 - ii. Fills a table with solutions to subproblems in a bottom-up manner.
 - iii. Suitable when the number of overlapping subproblems is small and the optimal solution can be directly computed from the solutions to smaller subproblems.

Tabulation?

- Tabulation is a bottom-up approach to dynamic programming.
- Instead of solving the problem recursively, **it builds a table iteratively from the base cases up to the desired solution.**
- The idea is to **solve all subproblems** in a systematic manner and **store their results in a table** (usually an array) to **use them** in solving **larger subproblems**.

How Tabulation Works

1. **Initialize the Table:** A table (usually an array) is created to store the results of subproblems.
2. **Base Case:** The base cases are initialized directly in the table.
3. **Iterative Computation:** The table is filled iteratively, starting from the smallest subproblems and building up to the desired problem.
4. **Final Solution:** The solution to the original problem is found in the last cell of the table

e.g Tabulation

```
function Fibonacci(n):  
    table = array of size n+1  
    table[0] = 0  
    table[1] = 1  
    for i from 2 to n:  
        table[i] = table[i-1] + table[i-2]  
    return table[n]
```

Memoization vs Tabulation

Memoization

Top-down approach

Uses recursion

Stores results as they are computed

Solves only the subproblems that are needed

The key difference is how the values are computed:

1. Recursion: Memoization (top-down)
2. Iteration (loops): Tabulation (bottom-up)

****Note:** memoization may use an array or hash map to store results, but that does not make it tabulation.

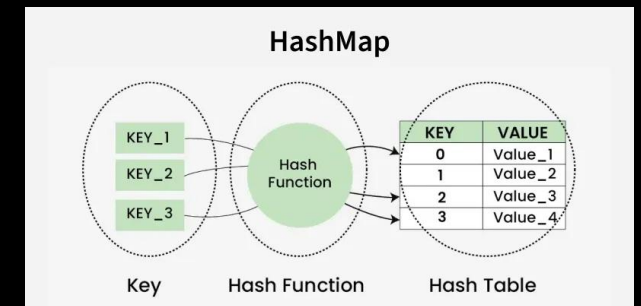
Tabulation

Bottom-up approach

Uses iteration (loops)

Fills a table from smallest to largest subproblem

Usually computes all subproblems

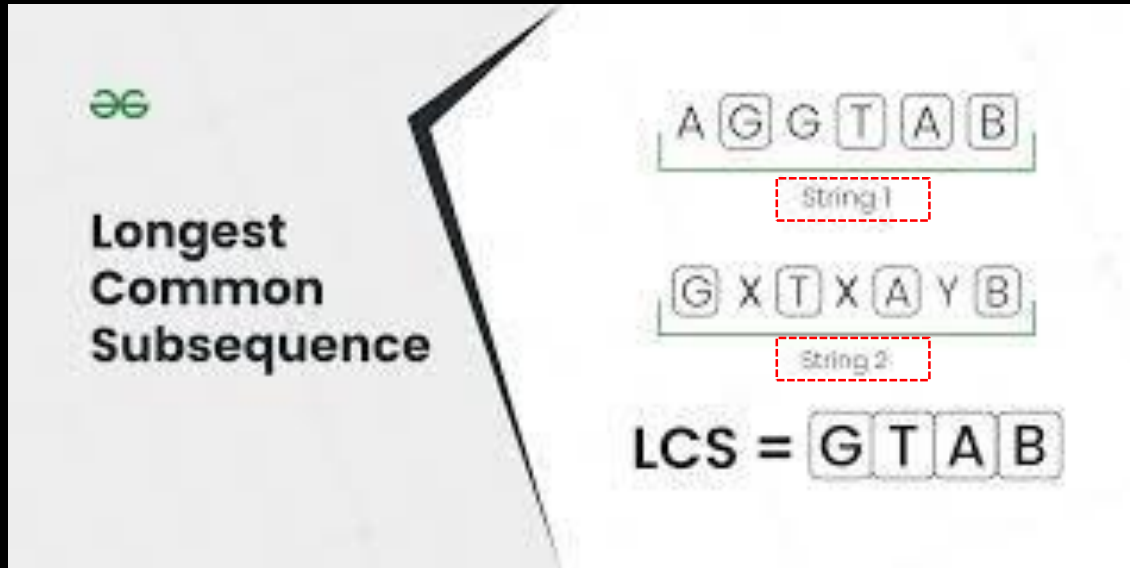


	Tabulation 	Memoization 
State	State Transition relation is difficult to think	State transition relation is easy to think
Code	Code gets complicated when lot of conditions are required	Code is easy and less complicated
Speed	Fast, as we directly access previous states from the table	Slow due to lot of recursive calls and return statements
Subproblem solving	If <u>all</u> subproblems must be solved at <u>least once</u> , a bottom-up dynamic-programming algorithm usually outperforms a top-down memoized algorithm by a constant factor	If <u>some subproblems in the subproblem space need not be solved at all</u> , the memoized solution has the advantage of <u>solving only those subproblems that are definitely required</u>
Table Entries	In Tabulated version, starting from the first entry, all entries are filled <u>one by one</u>	Unlike the Tabulated version, all entries of the lookup table are not necessarily filled in Memoized version. <u>The table is filled on demand.</u>

Types of Memoization: 1D, 2D and 3D

- The Implementation of memoization depends upon **the changing parameters that are responsible for solving the problem**.
- There are various dimensions of caching that are used in memoization technique, Below are some of them:
 - **1D Memoization**: The recursive function that has only **one** argument whose value was **not constant** after every function call.
 - e.g Fibonacci series
 - **2D Memoization**: The recursive function that has only **two** arguments whose value was **not constant** after every function call.
 - e.g computing the Length of computing the Length of LCS for **two strings**
 - **3D Memoization**: The recursive function that has only **three** arguments whose values were **not constant** after every function call.
 - e.g computing the Length of LCS for **three strings**

What is LCS referred in previous slide?



"gxtxayb"
"abgtab"
"gyaytahjb"

The LCS is shown using red characters in the 3 given strings.

Three Stings Example

string 1	a	c	b	a	e	d
string 2	a	b	c	a	d	f

LCS: "acad" with length 4

Two Stings Examples

```
#include <iostream>
#include <unordered_map>
```

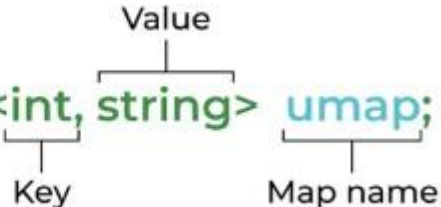
Memoization implementation

```
int fibonacci(int n, std::unordered_map<int, int>& cache) {
    if (cache.find(n) != cache.end()) {
        return cache[n];
    }
    int result;
    if (n == 0) {
        result = 0;
    } else if (n == 1) {
        result = 1;
    } else {
        result = fibonacci(n-1, cache) + fibonacci(n-2, cache);
    }
    cache[n] = result;
    return result;
}
```



Memoization (top-down cache filling)

```
unordered_map<int, string> umap;
```



https://www.geeksforgeeks.org/unordered_map-in-cpp-stl/

```
#include <iostream>
#include <vector>
```

Tabulation implementation:

```
int fibonacci(int n)
{
    if (n == 0) {
        return 0;
    }
    else if (n == 1) {
        return 1;
    }
    else {
        std::vector<int> table(n + 1, 0);
        table[0] = 0;
        table[1] = 1;
        for (int i = 2; i <= n; i++) {
            table[i] = table[i - 1] + table[i - 2];
        }
        return table[n];
    }
}
```



Tabulation (bottom-up cache filling)

<https://www.geeksforgeeks.org/tabulation-vs-memoization/>

unordered_map

- The unordered_map is a container in the C++ Standard Template Library (STL) that provides an associative array functionality.
- It stores elements in key-value pairs, where each key is unique and maps to a corresponding value.
- An associative array, also known as a map, dictionary, or hash table in various programming languages, is a data structure that stores data as a collection of key-value pairs.
- In the context of a phone book, an associative array is an ideal choice for organizing contact information due to its ability to efficiently store and retrieve data based on meaningful identifiers.

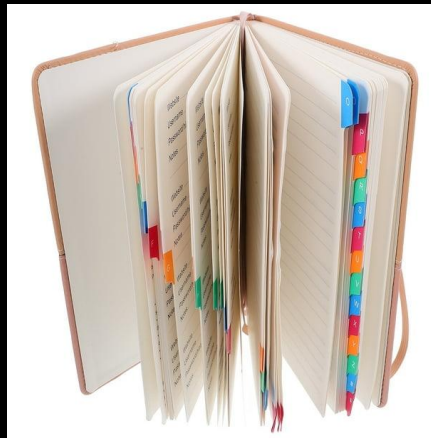


Diagram illustrating the components of the `unordered_map` template:

```
unordered_map<int, string> umap;
```

The diagram shows the template arguments `int` and `string` grouped by a bracket labeled "Key" and "Value" respectively. The variable `umap` is labeled "Map name".

```
#include <iostream>
#include <unordered_map>
#include <string>

int main() {
    // Declare an unordered_map with string keys and int values
    std::unordered_map<std::string, int> ages;

    // Insert elements
    ages["Alice"] = 30;
    ages["Bob"] = 25;
    ages["Charlie"] = 35;

    // Access elements
    std::cout << "Alice's age: " << ages["Alice"] << std::endl;

    // Update an element
    ages["Bob"] = 26;
```

Tabulation vs Memoization

- In above examples
 - The **memoization** implementation, we use a **dictionary object** called *cache* to store the results of function calls, and we use **recursion** to compute the results.
 - The **tabulation** implementation, we use an **array** called *table* to store the results of subproblems, and we use **iteration** to compute the results.
- Both implementations achieve the **same result**, but the **approach used is different**.
- **Memoization** is a top-down approach that uses **recursion**, while **tabulation** is a bottom-up approach that uses **iteration**.

Calculating Fibonacci sequence using DP

- Dynamic Programming can be applied to efficiently calculate the Fibonacci sequence:
 1. **Identify the subproblem:** The subproblem in this case is calculating the Fibonacci number at a specific index.
 2. **Build the solution incrementally:** Start by solving the subproblems from the smallest index upwards. Store the solutions in an array or table for future use.
 3. **Utilize the stored solutions:** When calculating a Fibonacci number at a particular index, retrieve the solutions of its preceding Fibonacci numbers from the table and use them to calculate the current Fibonacci number.

Calculating Fibonacci sequence using DP

- By storing and reusing the solutions to subproblems, Dynamic Programming **avoids redundant computations and greatly improves the efficiency** of calculating Fibonacci numbers.
 - For example, to find the Fibonacci number at index 5, the Dynamic Programming approach would first calculate the Fibonacci numbers at indices 0, 1, 2, 3, and 4, storing them in a table.
 - Then, using these pre-computed values, it can directly calculate the **Fibonacci number at index 5 without recalculating the entire sequence**.
- Dynamic Programming significantly reduces the time complexity from exponential ($O(2^n)$) in a naive recursive approach to linear ($O(n)$) in the Fibonacci sequence calculation.

Tabulation

- Tabulation is a **bottom-up approach** where we store the results of the subproblems in a table and use these results to solve larger subproblems until we solve the entire problem.
- Tabulation is **typically implemented using iteration** and **is well-suited for problems that have a large set of inputs**.

Some Advantages and Disadvantages of DP?

- Advantages of Dynamic Programming:
 1. Optimal solutions.
 2. Efficiency through avoiding redundant computations.
 3. Problem simplification through breaking down complex problems.
- Disadvantages of Dynamic Programming :
 1. Increased memory usage.
 2. Complexity in implementation.
 3. Dependency on problem structure.
 4. Computation overhead.

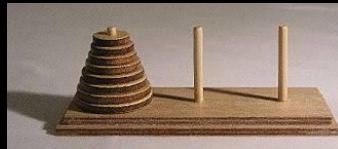
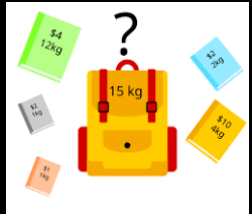
<https://testbook.com/key-differences/difference-between-divide-and-conquer-and-dynamic-programming>

Example Applications of DP

- Google Maps: shortest path computations
- Spellcheckers / DNA sequence alignment: edit distance
- Stock trading algorithms: optimal buy/sell timing
- Compilers: optimal register allocation
- Resource allocation: knapsack-style budgeting

Common Algorithms that Use Dynamic Programming:

- i. Longest Common Subsequence (LCS): Finds the longest common subsequence between two strings.
- ii. Shortest Path in a Graph: Finds the shortest path between two nodes in a graph.
 - All pair shortest path by Floyd-Warshall
 - Shortest path by Dijkstra
- iii. Knapsack Problem: Determines the maximum value of items that can be placed in a knapsack with a given capacity.
- iv. Matrix Chain Multiplication: Optimizes the order of matrix multiplication to minimize the number of operations.
- v. Fibonacci Sequence: Calculates the n^{th} Fibonacci number.
- vi. Tower of Hanoi
- vii. Project scheduling



What are the signs of DP suitability?

Identifying whether a problem is suitable for solving with dynamic programming (DP) involves recognizing certain signs or characteristics that suggest DP could be an effective approach. Here are some common signs that indicate a problem might be a good fit for dynamic programming:

1. **Overlapping Subproblems:** A problem that can be broken down into smaller subproblems that are solved independently, and the same subproblems encountered multiple times strongly indicates DP suitability.
2. **Optimal Substructure:** Problems that exhibit optimal substructure can often be solved using DP. This means that the optimal solution for a larger problem can be constructed from the optimal solutions of its smaller subproblems.
3. **Recursive Nature:** Problems that can be naturally expressed using recursion are often well-suited for DP.

What are the signs of DP suitability? Ctd..

4. Memoization Opportunities: If you notice that you can improve a recursive algorithm by **memoizing (caching) intermediate results**, DP might be a good fit.
5. Sequential Dependencies: Problems where the **solution depends on the results of previous steps or stages** are often candidates for DP. DP is particularly useful when solving problems involving sequences, such as strings, arrays, or graphs.
6. Optimization or Counting: DP is **often applied to optimization problems (maximizing or minimizing a value) or counting problems (finding the number of possible solutions)**.

Some FAQ

- Is memoization the same as caching?
 - Memoization is actually a specific type of caching. The term caching can generally refer to any storing technique (like HTTP caching) for future use, but memoizing refers more **specifically to caching function that returns the value.**
- Why memoization is top-down?
 - The top-Down approach breaks the large problem into **multiple subproblems. if the subproblem is solved already then reuse the answer. Otherwise, Solve the subproblem and store the result in some memory.**
- Where is memoization used?
 - Memoization is an optimization technique used to speed up by caching the results of **expensive function calls and returning them when the same inputs are encountered again.**

FAQ

- Should I use tabulation or memoization?
 - For problems requiring all subproblems to be solved, tabulation typically outperforms memoization **by a constant factor**.
 - This is because the tabulation has no overhead of recursion which reduces the time for resolving the recursion call stack from the stack memory.
 - Whenever a subproblem needs to be solved for the original problem to be solved, memoization is preferable since a subproblem is **solved lazily, i.e. only the computations that are required are carried out**.

FAQ

- Why tabulation is faster than memoization?
 - Tabulation is **usually faster than memoization**, because it is **iterative** and **solving subproblems requires no overhead of recursive calls**.
 - Memoization is a technique used to **speed up the execution of recursive or computationally expensive functions by caching the results of function calls** and **returning the cached results when the same inputs occur again**.
 - The basic idea of memoization is to store the output of a function for a given set of inputs and return the cached result if the function is called again with the same inputs.
 - This **technique can save computation time**, especially for functions that are **called frequently or have a high time complexity**.

- Refer : [MIT. Dynamic Programming](https://www.youtube.com/watch?v=OQ5jsbhAv_M&list=PLfMspJ0TLR5HRFu2kLh3U4mvStMO8QURm&index=2)
https://www.youtube.com/watch?v=OQ5jsbhAv_M&list=PLfMspJ0TLR5HRFu2kLh3U4mvStMO8QURm&index=2